



A novel approach to accelerate calibration process of a k -nearest neighbours classifier using GPU



Amreek Singh^{a,b,*}, Kusum Deep^a, Pallavi Grover^b

^a Indian Institute of Technology—Roorkee, Roorkee-247667, India

^b Snow & Avalanche Study Establishment, Chandigarh-160036, India

HIGHLIGHTS

- The SIMD model of parallel computing fitted into calibration sub-processes.
- A methodology formulated around GPU hardware architecture and memory hierarchy.
- Combines primitives of parallel implementations of ABC algorithm and k -NN algorithm.
- NVIDIA Tesla C2050 GPU used with CUDA programming framework.
- Over 10× acceleration achieved in calibration process.

ARTICLE INFO

Article history:

Received 11 December 2015

Received in revised form

30 December 2016

Accepted 5 January 2017

Available online 16 January 2017

Keywords:

NVIDIA CUDA

GPU

SIMD

ABC algorithm

k -nearest neighbours

Avalanche forecasting

ABSTRACT

General purpose data parallel computing with graphical processing unit (GPU) is much structured today with NVIDIA® CUDA and other parallel programming frameworks. Exploiting the CUDA programming framework, the present work proposes a novel methodology formulated around the GPU hardware architecture and memory hierarchy to accelerate the calibration process of a classification model named *eNN10*. Primarily developed for avalanche forecasting, *eNN10* is based on brute force k -nearest neighbours (k -NN) approach and employs snow-meteorological variables to search for past days with similar conditions. The events associated with past similar days are then analysed to generate forecast. The model is required to be calibrated regularly to ensure higher degree of forecast accuracy in terms of Heidke skill score (HSS). The calibration of *eNN10* is carried out by Artificial Bee Colony (ABC) algorithm, a swarm intelligence driven population based metaheuristic algorithm, and it requires thousands of HSS evaluations during the complete calibration process. A MATLAB sequential code for calibration runs for over 400 minutes and the proposed methodology delivered about 10× acceleration in calibration process. The methodology combines primitives of parallel implementations of brute force k -NN algorithm with that of population based metaheuristic algorithms and is scalable to deal with other similar real-world problems. The major objective of this paper is to highlight the methodology and associated future research areas.

© 2017 Elsevier Inc. All rights reserved.

1. Introduction

k -nearest neighbours (k -NN) is a popular non-parametric method for data exploration and classification. Snow & Avalanche Study Establishment (SASE) of India developed a k -NN based classification model, named as *eNN10*, for forecasting of snow avalanches in Indian Himalaya [49–51]. An avalanche is rapid flow of snow mass down the slope of a mountain. It can occur

due to several reasons ranging from increased load on snowpack due to new snowfall and metamorphic changes in snowpack to natural causes like rains, earthquakes, rockfall, etc. As an avalanche poses serious threat to lives of people living/venturing in the area, its forecast is of extreme importance. The information about past avalanche occurrences associated with nearest neighbours identified by *eNN10* model leads to forecast of future event (occurrence or non-occurrence of avalanche). However, in order to achieve high forecast accuracy from k -NN based models, certain model parameters are required to be pre-determined as emphasized by many researchers [8,17,46]. Mathematically it is an optimization problem and the act of determining optimal values of these model parameters is termed as model calibration [13].

* Corresponding author at: Snow & Avalanche Study Establishment, Chandigarh-160036, India.

E-mail address: amreek@sase.drdo.in (A. Singh).

Singh et al. [49] suggested that non-deterministic population based optimization and search algorithms known as metaheuristics [19,5,53] are particularly suitable for calibration of k -NN based models. Population based metaheuristic algorithms make iterative progress in a population through a guided random search to achieve the desired end and incorporate mechanisms to avoid getting trapped in confined areas of the search space. Singh et al. [49] applied Artificial Bee Colony (ABC) algorithm [24,25], a popular population based metaheuristic algorithm inspired by intelligence of honey bee swarms while foraging for food, to calibrate $eNN10$ model for two study areas of Indian Himalaya and obtained good results. However, the process of $eNN10$ calibration is computationally costly (associated MATLAB sequential program runs for about 400 min for each study area). It is attributed to thousands of evaluations of compute intensive objective function with respect to potential solutions during the calibration process. Under the present work we implemented general purpose Graphical Processing Unit (GPU) to speed-up the calibration process of $eNN10$ model by about $10\times$. This was achieved through a smart parallel computing methodology formulated around the GPU hardware architecture and memory hierarchy using NVIDIA® CUDA programming framework.

The paper is divided into 9 major sections. Section 2 describes $eNN10$ model in detail and discusses the complexity of its calibration process. Section 3 explains the ABC algorithm. Section 4 highlights the computational issue related to $eNN10$ calibration. Section 5 briefly discusses about GPU used and NVIDIA® CUDA programming model for code parallelization. Section 6 presents a review of related work by other researchers in the past. Section 7 explains the proposed parallel computing methodology as how the features of GPU hardware and CUDA programming model have been exploited to speed up the $eNN10$ calibration process. Results obtained have been discussed in Section 8. At last Section 9 summarizes the outcomes of this study and touches upon the limitations and future research scope.

2. $eNN10$ model for avalanche forecasting and its calibration

2.1. $eNN10$ model

For avalanche forecasting, k -NN algorithm was first introduced by Obled and Good [43]. Since then many variants of k -NN algorithm based models have been proposed for avalanche forecasting [7,8,28,35,17,6,16,36,46,21,51,50]. A k -NN algorithm deploys a distance metric to determine nearest neighbours of a given query from the reference dataset e.g. McClung and Tweedy [35] used Mahalanobis distance metric which has implicit features to deal with correlated variables. In this regard, $eNN10$ model uses Euclidean distance metric following Buser [7], Buser et al. [8] and many other researchers. Euclidean distance metric assumes no correlation between data variables. Table 1 describes the 10 snow-meteorological variables of $eNN10$ model. Invariably, Euclidean distance metric based k -NN models also apply distinct weights to data variables. For D -dimensional vector-space, the weighted Euclidean distance $d_{i,j}$ between any two data vectors $\mathbf{x}^i$ ($x_1^i, x_2^i, \dots, x_D^i$) and $\mathbf{x}^j$ ($x_1^j, x_2^j, \dots, x_D^j$) is calculated as—

$$d_{i,j} = \sqrt{\sum_{m=1}^D w_m \left(\text{norm}(x_m^i) - \text{norm}(x_m^j) \right)^2} \quad (1)$$

where,

w_m : weight corresponding to variable $m \in \{1, \dots, D\}$

$\text{norm}(\cdot)$: value of the variable normalized by its range in the database.

Further, the following steps have been implemented to generate forecast for any query $\mathbf{x}^i$ —

Table 1

Snow-meteorological variables of $eNN10$ model.

No. (m)	Variable (x_m)	Unit
1	Snow surface temperature	°C
2	Air temperature	°C
3	Air temperature change from previous day	°C
4	Wind speed (averaged over last 24 h)	m/s
5	New snow in 24 h	m
6	New snow in 48 h	m
7	New snow in 72 h	m
8	Snowpack depth	m
9	Snowpack water equivalent	m
10	Free penetration from surface	m

1. Calculate distances $d_{i,j}$, between query $\mathbf{x}^i$ and reference data points $\mathbf{x}^j$, $j \in \{1, \dots, n\}$ using Eq. (1), where n is the size of reference dataset,
2. Identify k smallest $d_{i,j}$ values (generally $k = 10$),
3. Corresponding to k smallest $d_{i,j}$ values, segregate data points (call ' k -nearest neighbours') from reference dataset,
4. Note events (avalanche/no-avalanche) associated with each of k -nearest neighbours,
5. Calculate probability $P_a(i) (= \frac{k_a}{k})$ of avalanche occurrence corresponding to query $\mathbf{x}^i$, where k_a is no. of avalanche cases out of k -nearest neighbours.
6. If $P_a(i) \geq P_t$, where P_t is a threshold value, then forecast with respect to query $\mathbf{x}^i$ is avalanche, else no-avalanche.

The above-described exhaustive procedure is called brute-force (BF) k -NN approach. For avalanche forecasting, a commonly adopted value of P_t is 0.3 [8]. It is obvious that with $k = 10$, $P_a \in \{0, 0.1, 0.2, \dots, 1.0\}$.

2.2. Model calibration—an optimization problem

In order to achieve high forecast accuracy from $eNN10$ (or any k -NN model), it is important to calibrate the model and fix optimal values of model parameters $\mathbf{w}$ ($w_1, w_2, \dots, w_D$). Singh et al. [49] treated the $eNN10$ model calibration as an optimization problem and used widely accepted Heidke skill score [58,21] to assess the forecast accuracy of the model. Corresponding to distribution of forecasts and observations (Table 2), Heidke skill score (HSS) can be defined as—

$$HSS = \frac{ad - bc}{((a + c) \cdot (c + d) + (a + b) \cdot (b + d)) / 2} \quad (2)$$

HSS as defined above is based upon the 'Percent Correct' $\left[= \frac{(a+d)}{N} \right]$ as the basic accuracy measure with 'random forecast' $\left[= \left(\frac{a+b}{N} \right) \cdot \left(\frac{a+c}{N} \right) + \left(\frac{d+c}{N} \right) \cdot \left(\frac{d+b}{N} \right) \right]$ as reference forecast. HSS has following main properties—

- $HSS = 1$: Perfect forecast
- $HSS > 0$: Skillful, better than reference
- $HSS < 0$: Less skillful than reference.

Thus, $eNN10$ model calibration is a 10-dimensional optimization problem with the objective to determine vector $\mathbf{w}$ such that HSS is maximized. In optimization problem parlance, HSS and $\mathbf{w}$ would be called objective function and decision variables vector respectively. The following constraints are applicable on decision variables in context of $eNN10$ calibration—

- (i) Inequality constraints: None
- (ii) Equality constraints: None
- (iii) Variable limits: $0 \leq w_k \leq 1$, $k \in \{1, \dots, D\}$, $D = 10$.

Table 2
Contingency table depicting the distribution of forecasted versus observed events.

		Forecast		
		Yes	No	
Observation	Yes	Hits (a)	Misses (c)	Observed 'yes' (a + c)
	No	False alarm (b)	Correct negatives (d)	Observed 'no' (b + d)
		Forecasted 'yes' (a + b)	Forecasted 'no' (c + d)	Total (N = a + b + c + d)

```

1  for i = 1 to n           // Query level
2    for j = 1 to n; j ≠ i  // Reference dataset level
3      Calculate  $d_{i,j}$  using Eq. (1)
4    end
5    Identify  $k$ -nearest neighbours
6    Calculate  $P_d(i)$ 
7    Generate  $forecast(i)$  using  $P_i$ 
8  end
9  Generate contingency table (Refer Table 2)
10 Calculate HSS using Eq. (2)

```

Fig. 1. HSS evaluation by leave-one-out cross-validation of forecasts (n represents number of data points in the dataset).

2.3. Evaluating the objective function

During the process of model calibration the objective function HSS is evaluated many times corresponding to potential solutions $\mathbf{w}$. To evaluate HSS, leave-one-out cross-validation [27] of model forecasts is carried out. As the name suggests, under leave-one-out cross-validation one by one each data point of the dataset serves as the *query* and the remaining dataset serves as the *reference* for that *query*. The complete procedure is depicted in Fig. 1. Thus for a dataset with n data points, there are n *queries* to be processed for each HSS evaluation, requiring $n(n - 1)$ weighted distance calculations.

2.4. Dataset

The *eNN10* data structure is defined by 10 variables as described in Table 1 along with a class type—yes/no referring to occurrence/non-occurrence of avalanche(s) on the day of data observation. The database used in this study corresponds to the period from year 1999 to 2012 (covering the winter seasons i.e. for the months from November to April only) for two climatologically diverse study areas in western Himalaya. Only complete data points with values available for all the variables have been considered. There are 1652 data points in the dataset corresponding to the first study area and 1622 data points corresponding to the second study area.

3. ABC algorithm

Singh et al. [49] deployed Artificial Bee Colony (ABC) algorithm to calibrate *eNN10* model to ensure maximized forecast accuracy for two study areas. Under this exercise the appropriate values of $\mathbf{w}$ ($w_1, w_2, \dots, w_D$) were sought through the ABC algorithm, while k and P_i continued to carry fixed values at 10 and 0.3 respectively.

ABC algorithm was introduced by Karaboga [24]. It mimics the food foraging behaviour of honey bee colonies to look for the best solution to solve an optimization problem. Each candidate solution is thought of as *food source* and a population of agents (colony of bees) is used to explore the search space. Each time an artificial bee visits a *food source*, it evaluates its quality (i.e. fitness

of objective function) and searches for a better *food source* in its neighbourhood. The information is shared with other bees.

The colony of artificial bees contains three groups of bees: *employee bees*—associated with specific *food sources*, *onlooker bees*—monitoring the progress of *employed bees* to choose a *food source*, and *scout bees*—selecting the *food sources* randomly. The number of *employed bee* is equal to number of *food sources*. *Onlooker bees* are also equal in number to *employed bees*. Following the bee groups, the algorithm works in three main phases.

Initially *food sources* are generated randomly. Assuming $\mathbf{w}^s = (w_1^s, w_2^s, \dots, w_D^s)$ as the s th *food source*, this operation can be defined as follows:

$$w_m^s = w_{m_min} + \text{rand}(0, 1)(w_{m_max} - w_{m_min}) \quad (3)$$

where, $s \in \{1, 2, \dots, SN\}$; $m \in \{1, 2, \dots, D\}$; SN is the number of *food sources* (it equals half of population size representing the total numbers of *employed bees* and *onlooker bees* put together) and D is the number of decision variables (problem dimensionality); $\text{rand}(0, 1)$ is a random number in the range 0, 1; and w_{m_max} , w_{m_min} are the upper and lower bounds of m th decision variable respectively.

Each *employed bee* is allocated a *food source*. It evaluates the quality of *food source* (i.e. objective function value) allocated to it and then searches around for better *food source*. An *employed bee* is allowed to search for better *food source* only once per iteration and the search direction is chosen randomly. To identify a new candidate *food source* $\mathbf{w}^{s'} = (w_1^{s'}, w_2^{s'}, \dots, w_D^{s'})$ around the old one $\mathbf{w}^s = (w_1^s, w_2^s, \dots, w_D^s)$, the following expression is used:

$$w_m^{s'} = w_m^s + \text{rand}(-1, 1)(w_m^s - w_m^u); \quad u \neq s \quad (4)$$

where $u \in \{1, 2, \dots, SN\}$ and $m \in \{1, 2, \dots, D\}$ are randomly chosen indexes; u is different from s ; and $\text{rand}(-1, 1)$ is a random number in the range $-1, 1$. If new *food source* is better, it replaces the old one in the memory of corresponding bee i.e. $w^s \leftarrow w^{s'}$. Otherwise, the old one is retained. This marks the end of *employed bees* phase. *Onlooker bees* are then deployed to various *food sources* memorized by *employed bees*, in numbers proportionate to probability $P(s)$ associated with each *food source* as defined below—

$$P(s) = \frac{\text{fit}(s)}{\sum_{s=1}^{SN} \text{fit}(s)} \quad (5)$$

where $\text{fit}(s)$ is the fitness value of s th *food source*. *Onlooker bees* then further search for better *food sources* around the *food sources* allocated to them by using Eq. (4). Again if a better *food source* is found, the old one is replaced by the new one in the memory of corresponding *onlooker bee*. With this ends the *onlooker bees* phase and also the first iteration (also called cycle) of algorithm. The best solution (position of best *food source*) is memorized. Successive iterations start with *employed bees* taking over the *food sources* memorized by *onlooker bees* of previous iteration.

If any *food source* remains unchanged for a predetermined number of iterations (called *limit*, a control parameter of ABC

Table 3

ABC algorithm control parameters for *eNN10* calibration experiments. $D (= 10)$ is the number of decision variables (problem dimensionality).

Control parameter	Criterion	Value
Population size	$D \times 6$	60
Number of <i>food sources</i> (SN)	(Population size)/2	30
<i>Limit</i>	$SN \times D$	300
Maximum Cycle Number (MCN)	Problem specific	1000

algorithm), the *scout bee* activates. The particular *food source* is abandoned and a new *food source* is selected randomly within the search space as per Eq. (3). Next iteration continues with one new *food source* selected by *scout bee*. Only one *scout bee* is allowed to activate per iteration.

It should be noted that the components of the new candidate *food source* w^s may violate the predefined boundary constraints of search space. A simple and popular method to tackle this problem is to set the violating components to the violated bounds, i.e.,

$$w_m^s = w_{m_min}, \quad \text{if } w_m^s < w_{m_min}$$

$$w_m^s = w_{m_max}, \quad \text{if } w_m^s > w_{m_max}. \quad (6)$$

The algorithm terminates according to the pre-defined termination criteria e.g. Maximum Cycle Numbers (MCN) permitted. The values of various control parameters set by Singh et al. [49] for ABC algorithm to calibrate *eNN10* are summarized in Table 3.

4. Computation issues

Singh et al. [49] calibrated the *eNN10* model with remarkable improvements in forecast accuracy in terms of *HSS* for the two study areas. Multiple experiments have been conducted to ensure stability of the results. The code is written in MATLAB for sequential computing by a workstation ($2 \times$ Intel® Xeon® $\times 56$ 602.80 GHz processor, 64 GB RAM, Windows 7, 64-bit OS). As per the control parameters set for calibration experiments (Table 3), objective function *HSS* is evaluated a little over 60 000 times before the termination of algorithm in each calibration experiment. It takes about 400 min for each experiment. Almost 98% of this compute time is consumed in objective function evaluations only (approximately 0.39 s per evaluation of *HSS*). The complexity of a *HSS* evaluation is: $O(n^2D)$ for the n^2 distances computed (approximately $2n^2D$ additions/subtractions and n^2D multiplications) and $O(nkn)$ for the n searches for k -nearest neighbour each, where n is the size of dataset in D -dimensional space.

In order to reduce the computational time of *eNN10* calibration process, it is proposed to implement GPU based parallel computing as GPUs are now well-established as powerful but low-cost platform for parallel computing. It does make sense as computing literature also suggests that the exhaustive BF k -NN is the most efficient alternative for GPU based metric-space *query* processing especially for high-dimensional metric-spaces and if quality of the solution cannot be compromised [3,9,31]. Moreover BF k -NN is an exact algorithm. There are several faster k -NN methods but with a trade-off between speed and accuracy [29,44,57]. If some amount of error can be tolerated, these algorithms may be used. However, the effect of faster algorithm will be significant when dataset is large. Thus a smaller dataset (about 1700 data points) coupled with the accuracy requirement of the solution renders the exhaustive BF k -NN algorithm a viable alternative for implementation of GPU based parallelism for *eNN10* calibration.

In order to implement the above proposal, a part of the code dealing with *HSS* evaluation has been re-written in accordance with NVIDIA® CUDA parallel programming framework and ensuring compatibility to run with NVIDIA® Tesla™ C2050 GPU device. This device follows Fermi micro-architecture developed by

Table 4

Key features of NVIDIA Tesla C2050 GPU.

Feature	Details
# of CUDA cores	448
Frequency of CUDA cores	1.15 GHz
ComputeCapability	'2.0'
DriverVersion	7.5000
MaxThreadsPerBlock	1024
MaxThreadBlockSize	[1024 1024 64]
MaxGridSize	[65535 65535]
SIMDWidth	32
MaxSharedMemoryPerBlock	49 152 B
TotalMemory	2.8186 GB
FreeMemory	2.5934 GB
MultiprocessorCount	14

NVIDIA [18]. The other key features of this GPU device are summarized in Table 4. A smart methodology has been formulated to overcome the limitations of programming framework as well as hardware architecture. This will be discussed in subsequent sections.

5. General purpose computation on GPU using CUDA

GPU is a specialized coprocessor that addresses the demands of real-time high-resolution 3D graphics and compute-intensive tasks. It allows floating point calculations much faster than CPU and beyond graphics applications. To put simply, GPU is an accelerator to a regular CPU. Unlike CPUs, GPUs have a parallel throughput architecture that emphasizes executing multiple concurrent *threads* slowly, rather than executing a single *thread* very quickly. The GPU is especially well suited to address an application if it fits both of the following criteria [47]:

Computationally intensive—The time spent on computation significantly exceeds the time spent on transferring data to and from GPU memory,

Massively parallel—The computations can be broken down into hundreds or thousands of independent units of work.

In other words, application should have scope for execution of same instructions on many data elements in parallel with a high ratio of arithmetic to memory operations. Applications that do not satisfy these criteria might actually run slower on a GPU than on a CPU.

Several general purpose high level frameworks like CUDA and OpenCL are available for GPU based parallel programming. They primarily follow **Single Instruction Multiple Data (SIMD)** model i.e. they execute same instruction on multiple data points simultaneously. The CUDA allows developers direct access to the virtual instruction set and memory of the parallel computational elements in NVIDIA® GPUs. With CUDA, the approach to use GPU for general purpose processing (i.e., not exclusively graphics) is known as GPGPU.

Under CUDA programming paradigm, the program is partitioned into many independent sub-tasks which can be solved in parallel. Each sub-task may be further divided into many elements, which can be solved cooperatively in parallel too. Each elemental task is run on a *thread* and all such cooperating *threads* running in parallel define a *thread block*. So many *thread blocks* can also run in parallel. The software segment that describes the instructions to be executed by each *thread block* and further by each *thread* of a *thread block* is called *kernel*. A program running on CPU invokes a *kernel* to assign a job to GPU. It is necessary to specify the number of corresponding *thread blocks* and the number of *threads* per *thread block* while invoking *kernel*. The array of the *thread blocks* is called *grid* and it can be one or two-dimensional i.e. each *thread block* within a *grid* has a unique set of indices for its identification. Similarly

the threads within a block can be organized as a one, two or three-dimensional array called *ThreadBlockSize*. The product of the elements of *ThreadBlockSize* must not exceed the *MaxThreadsPerBlock*, and no element of *ThreadBlockSize* can exceed the corresponding element of the GPU device property *MaxThreadBlockSize*. Preferably the number of threads per block should be a multiple of 32 threads, because this provides optimal computing efficiency and facilitates coalescing (NVIDIA 2014b). This mechanism allows the programmer to allocate personalized data to each thread and thus achieve effective problem decomposition. The key to obtain optimum results from GPU based computing is the exploitation of CUDA memory hierarchy as described below—

- (i) *Global memory*—is off-chip i.e. it resides in device memory. It is accessible by CPU as well as all threads of any kernel and has a data lifetime from allocation to de-allocation by host code. However, for devices of compute capability 2.x and higher, the global memory transactions are cached, so data locality is exploited to reduce impact on throughput [41].
- (ii) *Local memory*—is so named because its scope is local to the thread, not because of its physical location. In fact, local memory is also off-chip, hence access to local memory is as expensive as access to global memory [40]. On devices of compute capability 2.x and 3.x, local memory accesses are always cached in L1 and L2 in the same way as global memory accesses.
- (iii) *Shared memory*—is on-chip and has much higher bandwidth and lower latency than local and global memory. Shared memory is equivalent to a user-managed cache: The application explicitly allocates and accesses it. It is allocated per thread block and is accessible by any thread in the block. It has same lifetime as the thread block. Shared memory is expected to be much faster than global memory [41].

Thus a good CUDA program aims at minimizing data transfers between global memory and the device by maximizing use of on-chip memory i.e. shared memory and caches. Glaskowsky [18] and NVIDIA [40–42] provide a detailed information and guidance on Fermi compute architecture and CUDA programming model.

6. Related work

The present work involves two main algorithms: ABC and exhaustive BF k -NN. Both ABC and BF k -NN are very well-researched algorithms from accelerated computing point of view, as both the algorithms are well suited for cluster computing as well as GPGPU implementation. The parallel implementation of ABC algorithm is rather straightforward and follows general strategies applicable for parallel implementation of population based metaheuristic algorithms. A popular strategy is the master–slave configuration, where the population is split into subpopulations and distributed across the available processors. Narasimhan [39] was the earliest to present such a parallel version of the ABC algorithm for shared memory architectures. In this work the entire colony of bees is divided equally among the available processors. Each processor has a set of solutions in its local memory with a copy of the same in shared memory. During a neighbourhood search, a bee improves a solution in the local memory by randomly picking another solution from the shared memory. At the end of the cycle, the corresponding solutions in the shared memory are updated and made available to all other bees. The proposed parallelization strategy has been shown to have achieved a near linear speed-up with efficiency close to 1 and without degrading the quality of solutions obtained. Basturk and Akay [4] introduced improved and synchronous ABC algorithm (PAABC) using master–slave parallelization model through message passing interface (MPI). Hong et al. [22] also followed

master–slave approach to propose an MPI-based parallel ABC algorithm splitting the bees to each slave processes. Each slave process generates its own initial random solutions, subjects them to further bee actions, and directs the results to the main process.

Slightly varying from the above are parallel independent runs and multiple swarms approaches as implemented and analysed by Subotic et al. [54,55]. In parallel independent runs approach sets of small-size subpopulations are run in parallel in different compute processor or CUDA block. The independent processors have no communication between themselves at all. Every processor runs the same algorithm with different random seeds. If one subpopulation finds an acceptable solution first, then executions of all subpopulations are stopped. Alternatively, the final solution is the best one of all the independent runs. The idea of multiple swarms based approach is to use more than one swarm on the same search space. These swarms communicate with each other after certain number of cycles to exchange the results and the new solution matrix is formed in all swarms, based on best-so-far solutions from each swarm.

Another common approach is distribution of population members on independent computational resources for parallel processing. Gómez-Iglesias et al. [20] proposed an ABC adaptation to a grid computing environment for large-scale scientific optimization problems. The implementation distributes bees to different computational resources and coordinates the distributed execution of the system with some modifications in basic ABC algorithm. Muñoz et al. [37] also followed this approach to implement a hardware parallel architecture of the opposition-based ABC algorithm (HPOABC) in Field Programmable Gate Arrays (FPGAs). The basic idea of opposition-based learning (OBL) approach, first introduced by Al-Qunaieer et al. [1], is that whenever a solution is being explored in a direction, it is beneficial to consider the opposite direction as well. The OBL approach replaced the scout bee stage in this implementation. The food source update equations are computed in parallel through hardware resources.

Though no work is found in literature in respect of GPU based parallel implementation of ABC algorithm, substantial work has been carried out on many other metaheuristic algorithms such as Ant Colony Optimization [32,12,10], Genetic Algorithm [30,45], Particle swarm optimization [59,38], Cuckoo Search [23], Bees Algorithm [34], and so on. These GPGPU implementations suggest that the GPU can be used for parallel implementation of ABC algorithm as well.

Most of the reported parallel implementations of the BF k -NN approach have two main components: (1) to compute the distance matrix containing the distances between the query points and the reference points, and (2) to sort the distance matrix. Garcia et al. [14] proposed one of the earliest such GPU-based parallelization of the BF k -NN approach with two kernels where: (1) each thread computed the distance between a given query point and a reference point, and (2) each thread sorted all the distances computed for a given query point. An important aspect of this work was the use of an insertion sort algorithm which only outputs the k smallest elements. They reported a speed-up by up to two orders of magnitude for high-dimensional large dataset in comparison to the CPU based implementation. It may be noted that they used global memory for storing the query set (coalesced readings), and texture memory for the reference set (non-coalesced readings). It contributed to better performance in first part of the algorithm. However they used global memory for sorting part too. Since sorting consists in comparing and exchanging (reading and writing) many distances in a non-predictable order, the memory accesses are not coalesced. Therefore it penalized the sorting performance.

Following a slightly different strategy, Liang et al. [33] proposed CUKNN, a CUDA based parallel implementation of k -NN. Their implementation also performs two kernels: (1) a local k -NN

calculation for each *block* of *threads*, then (2) merging and sorting them in order to obtain a global k -NN. Memory coalescing technique has been used in distance calculation *kernel* to reduce the number of memory read transactions from the GPU *global memory* to the on-chip *shared memory*. Stream is also applied in this *kernel* so that the next memory read latency is hidden by the current computation. In sorting *kernel*, the distances between unknown object p and the training objects in a *block* are stored in the *shared memory*. Each *thread* owns one distance and obtains its rank by comparing it with other distances in the *shared memory*. Hence, the k -nearest neighbours of p in the *block* are obtained, called *local k -nearest neighbours*. The *global k -nearest neighbours* are then generated by a single *thread* from pool of all *local k* ones of an unknown tuple. Finally the *global k* nearest neighbours are transferred to the CPU to find out the majority labels of the *global k -nearest neighbours* and determine the label of the unknown object. However, this mechanism is suitable for small values of k (up to 20). Otherwise, CUKNN experiments showed good scalability on data objects as well as presented up to $15.2\times$ speedup in overall execution over a dataset of size 524 K.

Kuang and Zhao [29] used CUDA and proposed a GPU-based implementation of BF k -NN computation with *radix sort* [48] and achieved more than $10\times$ faster performance than the sequential counterpart. They utilized a segmentation method for pair-wise distance computation, which is inspired by the distance computation method presented in Chang et al. [11]. The algorithm uses one *thread* for one entry in $n \times n$ output matrix of distances for n input data points. Thus there are n^2 *threads* involved. The *threads* are organized into 16×16 two-dimensional *blocks*, and the *blocks* are organized into an $n/16 \times n/16$ two-dimensional grid. At a time each set of 16 *threads* in a *block* shares one copy of data in *shared memory* to calculate 16 distances and there are 16 such entries of data in *shared memory* (one for each of 16 sets). The smaller number of *threads* per *thread block* due to limitations of device (NVIDIA Tesla C870) highly increased the number of data movements. Also in the final performance test (123 features dataset including a train set of size 30956 and test set of size 1605) the sorting phase occupied the largest proportion of the overall computing time. Finally the classification labels of *query* points are determined on CPU as the many program branches made it difficult to optimize it for the GPU execution.

The GPU based BF k -NN algorithm proposed by Kato and Hosino [26] also has two parts: an N -body problem and a *partial sort*. It relied on the idea of a known algorithm for the *N-body problem in physics* along with another trick to overcome the problem of small sized *shared memory*. For the *partial sort*, each *thread block* is assigned a row of calculated distances. Only smallest k distances are computed by *heap sort* in parallel on each *thread* in the *thread block*. Use of *heap sort* is a smart choice for good performance since only top k elements are needed and not the fully sorted data. However still there is a room for improvement because it uses arrays with local scope which are stored in slow *global memory*.

Later Garcia et al. [15] proposed a newer GPU-based approach by using the CUBLAS (CUDA Basic Linear Algebra Subroutines) library for faster BF k -NN parallelization in order to efficiently calculate a distance matrix. The entire process was disintegrated into 8 parts, each of which is to be processed through a *kernel*. This arrangement leveraged implementation of some smart tricks to adopt coalesced read/write for faster computing and also to avoid unnecessary computations. Further if the distance matrix is too large to be handled by GPU memory, the *query* points are split, processed separately, and the distances to the k -nearest neighbours are merged together on the CPU/classical memory side.

Sun et al. [56] proposed a distributed approach for solving the k -NN problem on large instances. They proposed two layers

of parallelization: (1) an MPI based layer, distributing the data to several GPU enabled nodes, (2) CUDA layer, computing the k -nearest neighbours for each *query* point. Finally, all the results are combined in the merging step.

Barrientos et al. [3] also applied two-stage scheme to implement BF k -NN methods on GPU. First, all the distances from the target *query* to the elements on the database are evaluated and then a second stage sorts these distances to obtain the nearest elements. They exploited *coarse-grained* inter-query parallelism for dealing with multiple *queries* while *fine-grained* parallelism is offered when solving a single *query*. Overall, each *query* is processed by a different CUDA *block*. Their implementation of the sorting phase is based on a *logarithmic reduction* algorithm that consists of three nested levels. First, the distance vector is evenly distributed across the CUDA *block threads*. Each *thread* determines its own partial k -nearest neighbours by *heap sort*. The other two stages implement the reduction of the partial heaps. As per authors, heap insertions usually imply warp divergences and lack of locality, thus increasing the number of read/write operations.

Taking inspiration from the basic algorithm and *thread* organization of Chang et al. [11], Arefin et al. [2] proposed fast and scalable two-stage BF k -NN implementation GPU-FS-kNN using GPUs, where the distance matrix is divided into smaller sub-matrices (chunks) in order to parallelize distance calculations and k -NN search over these chunks. It can typically handle instances with over 1 million points and fairly larger values of k and dimensions (e.g., tested with k up to 64 and dimension up to 295) on a single GPU. In their *distance kernel*, each *thread* is responsible for computing a single distance in the matrix. Under the k -NN *kernel*, each *thread* works on a single row and identifies the k -nearest neighbours for respective row index. An interesting feature of the method is the introduction of padded input matrix rows and columns so that the algorithm can work with any number of rows and columns. According to authors, the main limitation of the implementation is its requirement to hold the complete dataset in the device memory.

Sismanis et al. [52] examined the relationship between two related basic operations, sort and select, and introduced a cohort of *truncated sort* algorithms for k -NN search. One first locates the k th element as a threshold, then searches all the elements smaller than the threshold, followed by another search to finish the k -list with elements equal to the threshold. The time complexity of parallel scan is $O(\log n)$.

Li and Amenta [31] claim to have presented the fastest BF k -NN implementation so far by using a modified inner loop of the SGEMM *kernel* in MAGMA¹ library, a well-optimized open source matrix multiplication *kernel*. Besides, they search only k smallest squared distances for each *query* by using the *merge-path* function from the Modern GPU library.²

As discussed above, there are certain limitations with most of the existing k -NN approaches e.g. the memory accesses are not coalesced or the method itself is not scalable to very large instances [14], the value of k is limited [33], the sorting phase is the bottleneck in performance of the whole application [29], relies more on *global memory* [26], invokes too many *kernels* per *query* thereby increasing host-device memory transactions or relying on CPU for some elementary parallel operations (e.g. Garcia et al. [15] merge the distances on CPU), sorting algorithm implying warp divergences and lack of locality [3], and device memory limitation [2]. Nevertheless these efforts provide a good insight which benefited us in many ways in parallel implementation of BF k -NN for specific purpose of eNN10 calibration.

¹ <http://icl.cs.utk.edu/magma/>.

² <http://nvlabs.github.io/moderngpu/>.

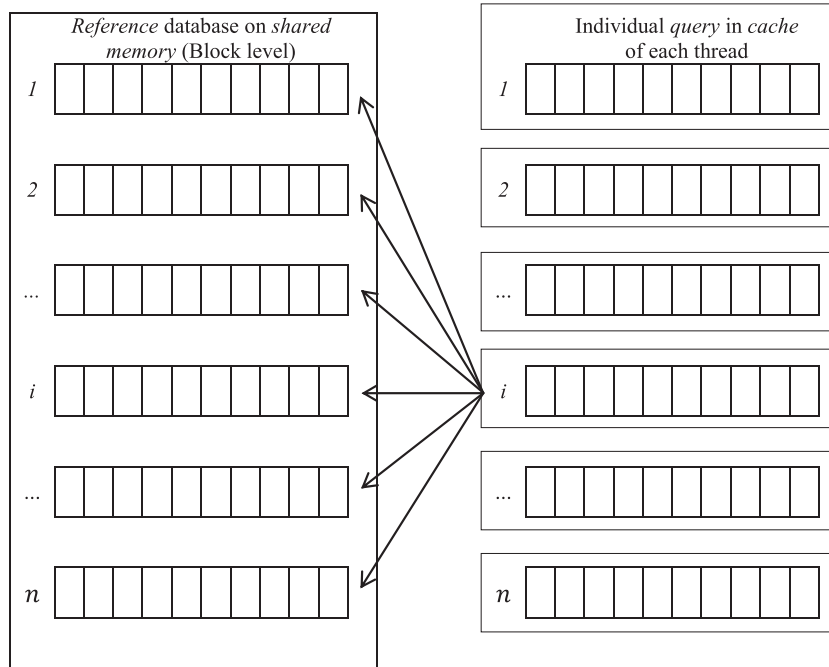


Fig. 2. Schematic of computing n^2 distance values under each HSS evaluation, where n distances are computed with respect to each of n queries. The distance between any query i and reference i is obviously 0.

7. Proposed approach

The proposed GPGPU implementation for *eNN10* calibration consists of two levels of parallelization: at top level a parallel ABC algorithm and at nested level a parallel BF k -NN algorithm i.e. multiple *food sources* are evaluated in parallel (*coarse grained* implementation), while an individual *food source* evaluation (involving k -NN algorithm) is also accelerated by parallel computing (*fine grained* implementation).

The evaluation of a *food source* in terms of HSS value requires processing of dataset for leave-one-out cross validation of forecasts (see Fig. 1). Apparently the leave-one-out cross validation is a typical SIMD process and well satisfies the criteria defined by Reese and Zaranek [47] for obtaining good results from GPU (see Section 5). A simple approach would be to assign one *query* per *thread* and let each *thread* compute n distances with all the n data points in the dataset (including the distance with itself, which is obviously 0, but in order to maintain the simplicity of algorithm). Under such arrangement, we may load individual *queries* on *cache memory* (through *local memory*) associated with *threads* to reduce memory transaction overhead and additionally copy the dataset on *shared memory* for faster access by each *thread* as depicted in Fig. 2. However, the number of data points is invariably higher than the *MaxThreadsPerBlock* (1024 in case of Tesla C2050 GPU used in this study). Hence a one-to-one mapping of data points with available *threads* per *thread block* is not always possible. Besides, the device has a *shared memory* capacity of 48 kB only. Thus keeping in view the features of GPU device (Refer Table 4) and that of CUDA programming model, following setting/restrictions are imposed for HSS evaluation on GPU—

GridSize: It is effectively the number of *thread blocks* that are launched independently by the GPU. In our methodology, this number is set as equal to number of *food sources* (SN). This arrangement ensures that fitness of all the *food sources* can be evaluated in parallel at any stage of the calibration process. It reduces the overheads due to data transfer between CPU and GPU and thus accelerates the process.

ThreadBlockSize: It is the size of a *thread block* on the *kernel*. In proposed approach, it is a 1-dimensional array and it is assigned a

value equal to half of size of *reference dataset*. We allow a maximum value of [896] ($=28 \times 32$) in compliance with *MaxThreadsPerBlock* and *MaxThreadBlockSize* properties of the device. This number has been arrived at keeping in view the *MaxSharedMemoryPerBlock* property of device (48 kB), as a set of data points equal in number to the *ThreadBlockSize* value is required to be loaded on *shared memory* per *block* along with other ancillary data. Table 5 depicts the details of total data loading on *shared memory* per *block* with above configuration.

With the above configuration, the complete two-level methodology is described next—

Level-I: This level runs sequentially on main CPU. Here all the basic information required is defined or fetched. This includes defining control parameters for ABC algorithm, fetching the dataset and calculating ranges of data variables (*maximum value–minimum value*). For this methodology the size of dataset (the number of data points) must be an even number. If not, a dummy data point is added to the dataset where all variables are assigned very high values. Further transposed data matrix (*size of dataset* $\times$ *number of variables*), data labels (*yes/no* converted to 1/0) vector and variable ranges vector are allocated to GPU *global memory* using 'gpuArray'. Transpose of data is required as GPU array stores data in the form of stack. ABC algorithm is launched. Random *food sources* are generated and copied to device memory. *ThreadBlockSize* and *GridSize* properties of the *kernel* are specified. ABC algorithm progresses. Customary three phases of ABC algorithm are executed iteratively and CUDA *kernel* (refer Level-II) is launched wherever quality of *food sources* is required to be evaluated. Results are recorded in respective files after the termination of ABC algorithm.

Level-II: The second and core logical segment of methodology deals with HSS evaluation through CUDA *kernel*. The CUDA code for this level is maintained in a file with '.cu' extension. The compilation of this code produces another file with '.ptx' extension. These two files together define the corresponding CUDA *kernel*. The CUDA code has been designed around the GPU hardware architecture and memory hierarchy to maximize the utilization of GPU resources. Methodology is formulated to process a dataset of size up to 1792 ($2 \times$ maximum of *ThreadBlockSize* i.e. 896). This

Table 5

Details of data variables required to be loaded on *shared memory* per block and total loading on *shared memory*. These details correspond to the assumption that *ThreadBlockSize* = 896 i.e. 896 data points (*queries*) per block will be processed in parallel at a time.

S.No.	Data variables (with dimensions) loaded on <i>shared memory</i> per block	Data type	Size per data element		Total size (in Bytes)
			In Bits	In Bytes	
1	shared_reference_data[896][10]	Single precision	32	4	35 840
2	shared_foods[10]	Single precision	32	4	40
3	shared_yesno[1792]	Unsigned integer	16	2	3 584
4	shared_decision[1792]	Unsigned integer	16	2	3 584
Total <i>shared memory</i> occupancy per block (in Bytes)					43 048

	1	2	3	...	i	...	n
1	0	$d_{2,1}$	$d_{3,1}$	...	$d_{i,1}$	...	$d_{n,1}$
2	$d_{1,2}$	0	$d_{3,2}$	...	$d_{i,2}$	...	$d_{n,2}$
3	$d_{1,3}$	$d_{2,3}$	0	...	$d_{i,3}$	...	$d_{n,3}$
...	...	...	...	...	...	...	...
j	$d_{1,j}$	$d_{2,j}$	$d_{3,j}$	...	$d_{i,j}$	...	$d_{n,j}$
...	...	...	...	...	...	...	...
n	$d_{1,n}$	$d_{2,n}$	$d_{3,n}$	...	$d_{i,n}$	...	0

Fig. 3a. n^2 distance values arranged in the form of $n \times n$ matrix (n may hold a maximum value of 1792). $d_{i,j}$ represents Euclidean distance between query x^i and reference data point x^j where $i, j = 1, 2, \dots, n$. All the diagonal values representing the Euclidean distance of a data point x^i with itself are 0.

	1	2	...	$n/2$
1	0	$d_{2,1}$	...	$d_{n/2,1}$
2	$d_{1,2}$	0	...	$d_{n/2,2}$
...	...	...	...	...
$n/2$	$d_{1,n/2}$	$d_{2,n/2}$	...	0
...	...	...	...	...
n	$d_{1,n}$	$d_{2,n}$	...	$d_{n/2,n}$
$n+1$	$d_{n/2+1,1}$	$d_{n/2+2,1}$	...	$d_{n,n/2+1}$
$n+2$	$d_{n/2+1,2}$	$d_{n/2+2,2}$	...	$d_{n,n/2+2}$
...	...	...	...	...
$n+n/2+1$	0	$d_{n/2+2,n/2+1}$	...	$d_{n,n/2+1}$
...	...	...	...	...
$2n$	$d_{n/2+1,n}$	$d_{n/2+2,n}$	...	0

Fig. 3b. n^2 distance values re-arranged in the form of $(2n) \times (n/2)$ matrix. This arrangement facilitates the determination of distance values in parallel using GPU.

has been made possible by arranging data loading on GPU *shared memory* in two parts and distance metric calculations in four parts respectively. As n^2 distance values are required to be determined to process n data points (n may hold a value up to 1792), we can arrange these values in the form of a $n \times n$ matrix as shown in Fig. 3a where diagonal elements of the matrix are all zero. In order to deploy GPU for determining the value of off-diagonal elements of this matrix, it is re-arranged in the form as depicted in Fig. 3b. This arrangement facilitates one-to-one mapping of columns of re-arranged matrix with number of *threads* per *thread block*. Fig. 4 illustrates *block level* view of HSS evaluation and an explanation is given below in six steps assuming that there are 1792 data points (maximum number permissible) in the *reference* dataset. Let us refer to i th data point as R_i .

Step-I: The first half (R^1) of dataset i.e. data points from R_0 through R_{895} are loaded on *shared memory* (S). Further R^1 data is also copied to *cache* associated with *threads*, one data point per *thread tx* of corresponding index. On each of 896 *threads* of *block*, 896 distances $d_{tx,j}$ ($tx, j = 0, 1, \dots, 895$) are calculated as per Eq. (1). *Threads* are synchronized at the end of this step. Now each

of 896 arrays of distances (with 896 elements in each array) are cached on contiguous memory space (L_{tx}) associated with *thread tx* as single variables and constant sized arrays are automatically saved in register space on the chip with almost no costs for read and write (source: internet).

Step-II: *Shared memory* is overwritten with second half (R^2) of dataset i.e. data points R_{896} through R_{1791} . Euclidean distances ($d_{tx,j+896}$) are calculated again on each *thread tx* using *query* data points (R^1) on *cache* and *reference* dataset (R^2) recently loaded on *shared memory*. The results are stored in the same contiguous *cache* memory space associated with each *thread*. Finally *threads* are synchronized.

Step-III: R^2 data is copied to *cache* of *threads* replacing the R^1 , such that *thread* with index tx gets data with index $tx + 896$. Now the distances ($d_{tx,j+2688}$) are calculated on each *thread tx* using *query* data points (R^2) on *cache* and *reference* datasets (R^2) present in *shared memory*. Results are stored in the contiguous *cache* memory area and *threads* are synchronized.

Step-IV: R^2 data present in *shared memory* is again replaced with R^1 and distances ($d_{tx,j+1792}$) are calculated on each *thread tx* using *queries* (R^2) on *cache* and *reference* data (R^1) on *shared memory*. New distance values are added to contiguous memory area and *threads* are synchronized.

Step-V: The above procedure produces a matrix of size 3584×896 as shown in Fig. 3b. In this matrix the elements (1, 1), (2, 2), ..., (896, 896) and (2689, 1), (2690, 2), ..., (3592, 896) are all zero, as these elements correspond to distance of respective data points with itself. For further processing, the 3584×896 matrix is assumed to be divided from the middle into two sub-matrices of size 1792×896 each. Now from each column of two sub-matrices, a heap of k smallest distances is created by simple scanning of the column array k -times (on each subsequent scan, previous minimum is not included), where each *thread* handles one column (number of *threads* in the *block* is equal to number of columns in each sub-matrix). This scheme of creating k -heap of smallest values saves considerable computation time as we are not interested in sorting the entire column. Now we have 2×896 such k -heaps corresponding to 2×896 *queries*. Further with respect to each *query*, k -NN are identified from original dataset, their class labels ('1' or '0') are read, probability P_a is evaluated and accordingly forecast (yes/no) is decided in parallel by 896 *threads* in two batches (one for each sub-matrix).

Step-VI: Finally a single *thread* is engaged to compare *query* forecasts with actual observations, generate contingency table and determine HSS value.

All the previous GPU based k -NN implementations process each *query* as a CUDA *block* (or multiple coordinated *blocks* by assigning a chunk of dataset to each *block*, if dataset is large). As a result, each *thread* determines one distance (between *query* and one *reference* data point). All the distances (calculated by the *threads*) are then collected and passed on to host. The sorting algorithm (to determine k -NN) is generally executed on host. In case of multiple *queries*, proportionate numbers of additional CUDA *blocks* are engaged for parallel processing. Our work differs here from all the previous works. Parallel ABC allows processing of multiple

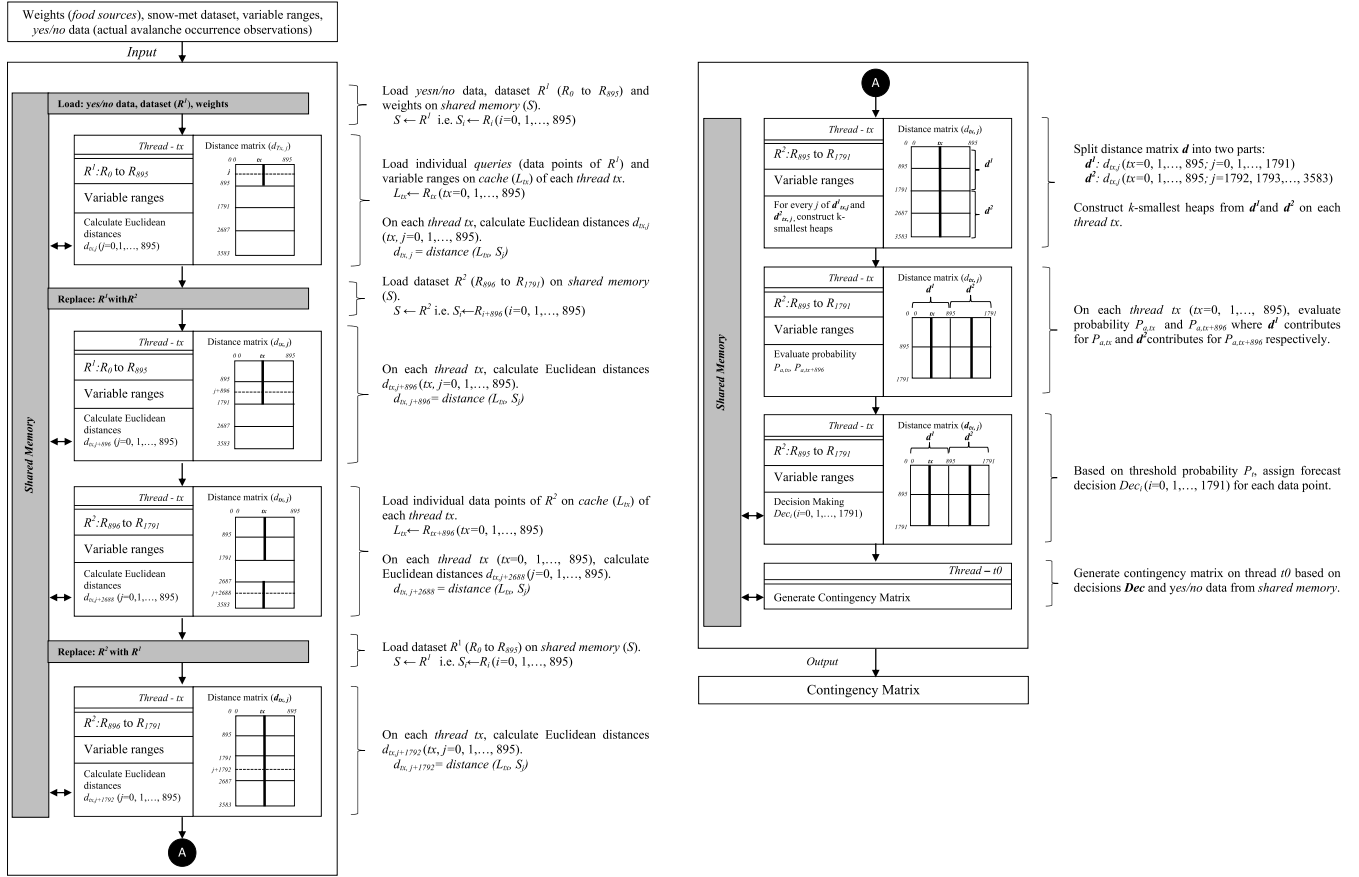


Fig. 4. Methodology to determine HSS using GPU with CUDA programming model.

instances of objective function evaluation by assigning a complete instance to a CUDA block (in contrast to other approaches, where only a single query is assigned to a CUDA block) and then each thread of the block determines all the n distances with respect to a query (in other approaches each thread determines just one distance). Further, since each thread maintains its own array of distances in cache memory, the k -NN are also identified by the same thread in continuation (in other approaches, k -NN are determined either on host or a second CUDA kernel is launched). These features reduce overhead due to data copy between host and device.

8. Results and discussion

In the *eNN10* calibration experiments carried out by Singh et al. [49], the datasets for two study areas have 1652 and 1622 data points respectively. In general a dataset with 1200 to 1800 data points is sufficient for implementing *eNN10* model for operational avalanche forecasting for any area. As already brought out, the proposed GPU based methodology is designed to process a dataset with up to 1792 data points. We repeated the calibration experiments of Singh et al. [49] for study area with 1622 data points in sequential compute mode (CPU-only single-core) and again in parallel compute mode (CPU-GPU implementation) with proposed methodology. The values of control parameters for ABC algorithm were also adopted from Singh et al. [49] as depicted in Table 3. Further, in order to ascertain the stability of proposed methodology, more experiments were conducted by varying values of control parameters—population size (*PopSize*) and *MCN* in such a way that overall value of *PopSize* $\times$ *MCN* remains constant.

8.1. Comparison with CPU-only single-core sequential implementation

The results of calibration experiments in terms of compute time of various segments of the process through CPU-only single-core implementation and through CPU-GPU implementation are summarized in Tables 6a and 6b respectively. Both the implementations follow the same code except for sub-process of HSS evaluations (refer to Level-II of previous section) which is processed in parallel on GPU in CPU-GPU implementation. Let us refer to this sub-process as *kernel* for further discussion purpose. The *kernel* is called many times during the progress of ABC algorithm under its various phases—initial population evaluation, employed bees phase, onlooker bee phase and scout bee phase. While in CPU-only serial implementation a *kernel* call will evaluate only one food source at a time, in CPU-GPU implementation a *kernel* call can evaluate multiple food sources also in parallel. Accordingly the numbers of *kernel* calls made by the algorithm under various phases of ABC algorithm are also summarized in respective results tables. In Table 6b reported times in Column C, E, G and I also include the overheads due to data transfer between device and host. Similarly the reported times in column K represent the total execution time (host + device) for one iteration of the ABC algorithm and column 'O' represents the total execution time (host + device) for a complete run of the ABC algorithm.

The comparison of Tables 6a and 6b reveals that CPU-GPU implementation has following advantages over the CPU-only serial implementation—

- The time consumed in a *kernel* call (under various phases of the algorithm) in CPU-GPU implementation is up to about 1/6th of the time consumed in similar operation under corresponding

Table 6a
Results of eNN10 calibration experiments in terms of compute time of various segments of the process through CPU-only single-core implementation. All time figures are in seconds. (Iter: Iteration, FEs: Function evaluations i.e. HSS evaluations.)

Pop size	Initial population evaluation		Employee bee phase		O/L bee phase		Scout bee phase		Total kernel calls ^a /Iter		Compute time /Iter	No of Iterations/Run	Total kernel calls/Run	Total FEs/Run	Total compute time/Run	HSS value
	kernel calls/Run	Compute time/kernel	kernel calls/Iter	Compute time/kernel	kernel calls/Run	Compute time/kernel	kernel calls/Iter	Compute time/kernel								
									A	B	C	D	E	F	G	H
10	5	0.439040	5	0.381349	5	0.392836	392	0.391387	10	3.871343	6000	60 397	23 645.21	60 397	0.40915	
20	10	0.423754	10	0.393967	10	0.388260	210	0.377743	20	8.222572	3000	60 220	24 639.29	60 220	0.41508	
30	15	0.410852	15	0.404668	15	0.390147	142	0.380068	30	11.940223	2000	60 157	24 635.01	60 157	0.41475	
40	20	0.406803	20	0.403875	20	0.387767	112	0.372362	40	16.378623	1500	60 132	24 476.91	60 132	0.41785	
50	25	0.402994	25	0.397245	25	0.407969	81	0.379195	50	20.096319	1200	60 106	24 495.48	60 106	0.42050	
60	30	0.402014	30	0.401495	30	0.397889	79	0.376589	60	23.969391	1000	60 109	25 126.48	60 109	0.42061	

^a Excludes scout bee phase calls.

Table 6b
Results of eNN10 calibration experiments in terms of compute time of various segments of the process through CPU–GPU implementation. All time figures are in seconds. (Iter: Iteration, FEs: Function evaluations i.e. HSS evaluations).

Pop size	Initial population evaluation		Employee bee phase		O/L bee phase		Scout bee phase		Total Kernel Calls/Iter		No of Iterations/Run	Total Kernel Calls/Run	Total FEs/Run	Total compute time ^c /Run	HSS value
	Kernel calls/Run	Compute time ^a /Kernel calls/Iter	Kernel calls/Iter	Compute time ^a /Kernel calls/Iter	Kernel calls/Iter	Compute time ^a /Kernel calls/Iter	Kernel calls/Run	Compute time ^a /Kernel calls/Run	Kernel Calls/Iter	Compute Time ^{b,c} /Iter					
A	B	C	D	E	F	G	H	I	J (D + F)	K	L	M (B + H + J*L)	N (A/2 + A*L + H)	O	P
10	1	0.128792	1	0.073893	5	0.072937	419	0.073004	6	0.440178	6000	36420	60424	2671.08	0.41428
20	1	0.131257	1	0.075944	10	0.072963	226	0.072980	11	0.807981	3000	33227	60236	2440.53	0.42013
30	1	0.207507	1	0.150857	15	0.072934	151	0.072900	16	1.248899	2000	32152	60166	2510.77	0.40435
40	1	0.207151	1	0.152045	20	0.073037	114	0.073194	21	1.616893	1500	31615	60134	2434.27	0.41527
50	1	0.209687	1	0.154659	25	0.072967	95	0.073149	26	1.985880	1200	31296	60120	2390.87	0.40741
60	1	0.283770	1	0.228740	30	0.073092	69	0.073061	31	2.426592	1000	31070	60099	2431.36	0.41520

^a Includes overheads due to data transaction between host and device.

^b Excludes scout bee phase calls.

^c Includes execution times on host and device, and overheads due to data transfers.

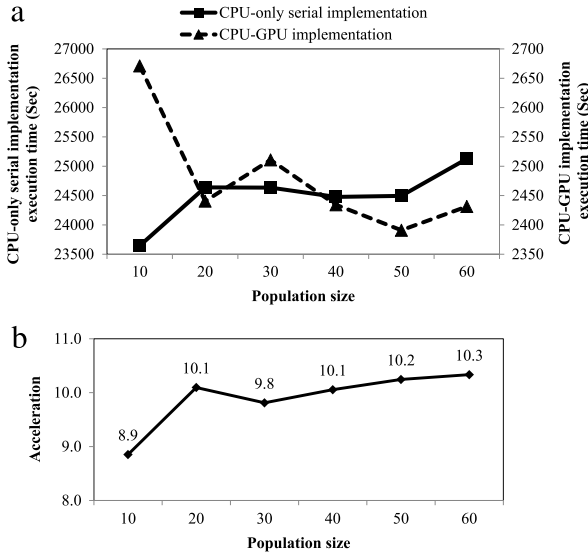


Fig. 5. (a) Calibration processing time (in seconds) with CPU–GPU parallel implementation in comparison to the processing time in CPU-only serial implementation for varying population sizes, (b) Acceleration achieved through CPU–GPU implementation with varying population sizes.

phase in CPU-only serial implementation (refer columns ‘G’ and ‘I’ of two tables).

- (ii) In CPU–GPU implementation a single *kernel* call can evaluate all *food sources* during *initial evaluation* phase and *employee bees* phase. Hence, the overall number of *kernel* calls is proportionately lesser in comparison to CPU-only serial implementation under corresponding phases (refer columns ‘B’ and ‘D’ of respective Tables).

As a result of above-mentioned advantages, a complete calibration experiment through CPU–GPU implementation consumes just about 1/10th of the time consumed through CPU-only serial implementation for similar experiment (refer column ‘O’ of two tables). This 10× acceleration takes into account all the overheads due to data transfer between host and device. Another important observation of these experiments is that the acceleration rate increases with increasing population size (refer Columns ‘O’ and Fig. 5(a), (b)) as the CPU-only serial implementation turns slower and CPU–GPU implementation turns faster with increasing population size. The fact that the final *HSS* values (refer columns ‘P’) obtained at the termination of each calibration experiment through both the compute implementations are not only close to each other but are also close to the values reported by Singh et al. [49], renders credibility to the results summarized in Tables 6a and 6b.

Further, a complete break-down of the execution compute time (refer column ‘O’ in Table 6b) is presented in Table 7. The Columns B, C and D of Table 7 are also depicted graphically in Fig. 6, where data copy time is presented on secondary axis while other two are on primary axis. Notably, in CPU–GPU implementation the main cost is data copy time (Column C of Table 7) which is about 99.5% of total execution time (Column E). Also all the three components of execution time gradually reduce with increasing population (refer to Table 7 and Fig. 6) as higher population size implies less number of iterations or *kernel* calls (reduces CPU compute time), larger *GridSize* (reduces GPU compute time) and less number of data copy instances (reduces data copy time) due to larger data packets involved.

In MATLAB, “gather” function is used to fetch the data from GPU to CPU after a *kernel* execution. The break-down of execution time in terms of various stages of an iteration of the calibration process with CPU–GPU implementation is summarized in Table 8 (for a population size of 60). For comparison, the execution time for

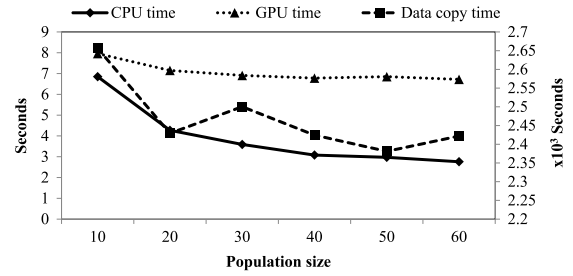


Fig. 6. Break-down of execution time of a calibration experiment through CPU–GPU implementation in terms of CPU time (primary axis), GPU time (primary axis) and data copy time (secondary axis) for varying population sizes.

same tasks with CPU-only serial implementation is also shown. The major beneficiary is *employed bee phase* with 55.1× acceleration. It is obvious that though an overall acceleration of about 10× has been achieved, it is largely restricted by data “gather” time (refer column F of Table 8). The GPU execution time (column E of Table 8 which also includes data copy time from host to device) is just about 0.2% to 0.3% of the data gather time (column F of Table 8). Thus any reduction in data gather time will substantially contribute to further speed-up. At this juncture there are following two possible ways to reduce data gather time, which may be used independently as well as in combination:-

- (i) Using a faster alternative to MATLAB “gather” function. This may also imply aborting the MATLAB-CUDA and exploring other GPU compatible programming platforms such as C/C++ OpenMP or openCL.
- (ii) Reducing the number of data transactions. By using a synchronous *onlooker bee* operator (as the case is with *employed bee* operator) number of data transfers may be further reduced. A preliminary analysis (detailed results not shown in paper) has revealed that the use of proposed synchronous *onlooker bee* operator can lead to overall about 50× speed up in calibration process through the same CPU–GPU-MATLAB-CUDA implementation. However, a separate detailed study will be required to ascertain if use of asynchronous *onlooker bee* operator leads to any significant downward effect on calibration results. This may be a subject of future research.

8.2. Comparison with CPU-only multi-core parallel implementation

Our host is a dual 6-core CPU i.e. 12 hardware cores. One may also think of accelerating calibration process by implementing a parallel version involving all CPU cores. If CPU–GPU implementation results in about 10× speed-up over a CPU-only single-core implementation then it is not wrong to expect that a CPU-only multi-core parallel solution might do just as well if not better. If that is not sufficient, then only deploying a GPU might be a good idea. Hence it is imperative to analyse the performance of a CPU-only multi-core parallel implementation of the algorithm.

As discussed earlier in Section 7, the problem of calibration of *k*-NN classifier has been addressed through two levels of parallel implementation:-

- (i) *Coarse-grained implementation*: It is especially useful during the *initial population* evaluation and *employed bee* phase of ABC algorithm, where multiple *food sources* are evaluated in parallel by as many independent CUDA blocks. In other phases of ABC algorithm (*onlooker bee* and *scout bee*) there is only one function evaluation required at a time, hence only one CUDA block is created.

Table 7

Break-down of calibration experiment execution time through CPU–GPU implementation. All time figures are in seconds.

Pop Size	CPU–GPU implementation				CPU-only serial implementation execution time	Acceleration
	Execution time on CPU	Data copy time	Execution time on GPU	Total execution time		
A	B	C	D	E (B + C + D)	F	G
10	6.85	2656.27	7.96	2671.08	23 645.21	8.9×
20	4.27	2429.11	7.15	2440.53	24 639.29	10.1×
30	3.50	2500.35	6.92	2510.77	24 635.01	9.8×
40	3.08	2424.41	6.78	2434.27	24 476.91	10.1×
50	2.97	2381.05	6.85	2390.87	24 495.48	10.2×
60	2.76	2421.88	6.72	2431.36	25 126.48	10.3×

Table 8

Break-down of execution time in terms of various stages of an iteration of calibration process with CPU–GPU implementation for a population size of 60. For comparison the execution time for the same tasks with CPU-only serial implementation is also shown. All time figures are in seconds. (FEs: Function evaluations).

Algorithm segments	CPU–GPU implementation						CPU-only serial implementation Total time/Iteration	Acceleration
	Kernel calls/Iteration	FEs/Kernel	FEs/Iteration	Execution time/Iteration	Data “gather” time ^a /Iteration	Total time/Iteration		
A	B	C	D	E	F	G (E + F)	H	I
Employed bee phase	1	30	30	0.000457	0.217966	0.218423	12.044850	55.1×
Onlooker bee phase	30	1	30	0.006930	2.161440	2.168370	12.044850	5.6×
Scout bee phase (if triggered)	1	1	1	0.000231	0.072048	0.072279	0.401495	5.6×
Complete iteration ^b	32	–	61	0.007618	2.451454	2.459072	24.491195	9.96×
Complete iteration ^c	31	–	60	0.007387	2.379406	2.386793	24.089700	10.09×

^a Data copy time from CPU to GPU is negligible and is inclusive in column E.^b With Scout Bee phase.^c Without Scout Bee phase.

(ii) *Fine-grained implementation*: It refers to objective function evaluation in parallel within a CUDA block. An objective function evaluation involves determination of n^2 distances where n is the number of data points. As all distance calculations are independent of each other, these can be determined in parallel. In proposed methodology, $n/2$ threads are allocated to a block and $2 \times n$ distances are calculated by each thread of the block. Further, each thread also identifies k -NNs from the corresponding set of distances generated by it i.e., $n/2$ processes for identification of $2 \times k$ -NNs run in parallel.

Thus it is an intricate combination of *coarse-grained* and *fine-grained* parallel implementations that defines our GPU based methodology and produces about $10\times$ speed up for the complete process of calibration for dataset used in the study. In this context, for CPU multi-core parallel computing MATLAB[®] offers *parfor* (parallel for) construct which is useful in situations that require many loop iterations of a set of calculations. To run *parfor* Parallel Computing Toolbox[™] is used. However, while implementing CPU-only multi-core version of algorithm for *eNN10* calibration problem, both *coarse-grained* and *fine-grained* parallel schemes cannot be integrated the way it is done in CUDA due to: (1) absence of any *block-thread* type architecture of GPU, and (2) the body of a *parfor*-loop cannot contain another *parfor*-loop.³ Hence, we conducted two experiments as described below involving a single iteration of calibration process (excluding the evaluation of initial population) for the same study area—

(i) Firstly under *coarse-grained* implementation, 30 food sources were assigned to 12 CPU cores for evaluation in parallel

through *parfor*-loop. Further at each core, a food source evaluation progressed sequentially (includes computation of n^2 distances and k -NN identifications).

(ii) In second experiment akin to *fine-grained* scheme 30 food sources were evaluated sequentially i.e., one food source evaluation at a time but n^2 distances and k -NN identifications under each food source evaluation were determined in parallel on 12 CPU cores using *parfor*-loop.

A *parfor* call launches MATLAB instances with private workspace on each worker (core). So, the data required for necessary calculations is sent from the client to workers, and the results are sent back to the client and pieced together. In our case also certain voluminous data of constant nature is required for necessary computations by each worker under every *parfor* call. MATLAB (version 2015b onwards) offers *parallel.pool.Constant* feature that copies the data values to each worker and allows the workers to access the data values within *parfor* construct. Appropriate implementation of this feature can improve performance when there are multiple *parfor* constructs accessing the same constant set of data, because data is transferred only once to the workers. Hence, we also implemented *parallel.pool.Constant* on data of constant nature (while some other data types of variable nature are still required to be copied on workers with every *parfor* call) in our multi-core experiments. The results of multi-core experiments in terms of compute time for *employed bee* phase and *onlooker bee* phase are summarized in Table 9 along with the performance results of other implementations (CPU single-core and CPU–GPU) for comparison. A discussion on these results is presented below—

Multi-core coarse-grain implementation: Under this implementation, *employed bee* phase turned approx. $3\times$ faster while *onlooker bee* phase turned approx. $3\times$ slower in comparison to corresponding segments in single-core implementation. Overall, the

³ <http://in.mathworks.com/help/distcomp/nesting-and-flow-in-parfor-loops.html>.

Table 9

Break-down of execution time of an iteration of model calibration process through CPU multi-core implementations with 12 cores. For comparison the break-down of execution time for the same task with CPU-only single-core and with CPU-GPU implementations are also shown. All time figures are in seconds. (FEs: Function evaluations).

Implementation type	Population size	Employed bee phase		Onlooker bee phase		FEs/Iteration	Compute time/Iteration	Overall acceleration
		FEs	Compute time	FEs	Compute time			
	A	B	C	D	E	F (B + D)	G	H
CPU-only single-core	60	30	12.1487	30	13.5570	60	25.7107	–
CPU–GPU	60	30	0.2316	30	2.1945	60	2.4304	10.58 ×
CPU-only multi-core (Coarse grained)	60	30	3.7797	30	37.6406	60	41.4228	(1/1.61) ×
CPU-only multi-core (Fine grained)	60	30	5.1105	30	5.2098	60	10.6045	2.42 ×

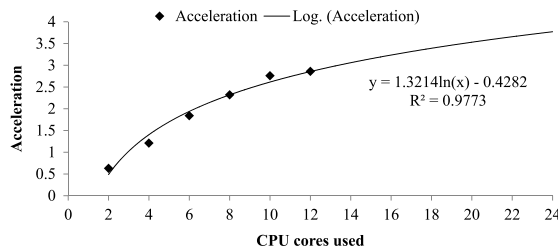


Fig. 7. The acceleration vs. CPU cores used as observed through CPU multi-core (combined) implementation.

multi-core coarse-grained implementation is $(1/1.61) \times$ slower in comparison to single-core implementation.

Multi-core fine-grain implementation: Under this implementation, both *employed bee* as well as *onlooker bee* phases got accelerated (by about $2.4 \times$ and $2.6 \times$ respectively) in comparison to corresponding segments in single-core implementation. Overall, the fine-grained implementation is $2.42 \times$ faster in comparison to single-core implementation.

Based on experience from above, in yet another experiment we combined coarse-grained and fine-grained implementations in such a way that *employed bee* phase undergoes coarse-grained parallelism while *onlooker bees* phase undergoes fine-grained parallelism. The results of combined multi-core implementation are presented in Table 10. The CPU multi-core (combined) implementation raised the computing acceleration level further to $2.86 \times$. However, it is still far lesser than $10.58 \times$ acceleration achieved through CPU-GPU implementation. Further, in order to study the effect of number of CPU cores used on acceleration rate, we conducted the CPU multi-core (combined) experiments with varying number of cores while keeping all the other conditions similar. The results so obtained are summarized in Table 11. The acceleration vs. number of CPU cores used is also presented in graphical form in Fig. 7. The acceleration grows algorithmically with number of CPU cores used. The projection suggests that hundreds of CPU cores will be required to match the acceleration achieved through CPU-GPU implementation, which is practically unviable. Besides there is also a possibility of an inflection point appearing in the acceleration curve at a certain stage.

9. Conclusions

Calibration of any forecasting model is important as a calibrated model ensures high degree of forecast accuracy. However calibrating *eNN10*, a classification model for avalanche forecasting based on brute force *k*-NN method, is a costly affair from computation point of view as it involves thousands of evaluations of a compute intensive objective function. The ABC algorithm used for *eNN10* calibration and the *k*-NN algorithm used in objective function evaluation are both well-suited for GPU based parallel implementations following the SIMD model. Hence the NVIDIA Tesla C2050 GPU governed by MATLAB-CUDA programming framework was

roped in to accelerate the process of *eNN10* calibration. A smart methodology formulated around the GPU hardware architecture and CUDA programming model delivered about $10 \times$ acceleration in *eNN10* calibration process. These results hold importance as CPU multi-core implementations for the same task did not yield encouraging results.

Other previous publications addressed the parallel implementations of *k*-NN algorithm and ABC algorithm separately. To our knowledge, this is the first work of its kind which combines primitives of parallel implementations of *k*-NN with population based metaheuristic algorithms. In this work, *k*-NN is at the core of objective function of calibration problem to be solved by ABC algorithm. Thus *k*-NN parallel implementation is nested to ABC parallel implementation. The methodology was tested with different parameter settings and results were found to be stable. The methodology can be ported on any other GPU of similar or higher specifications. Though the proposed methodology is restricted to process a dataset with up to 1792 data points only, it is large enough number to calibrate and run the model for operational purpose. Nevertheless, it will remain as our future research target to generalize the methodology to process larger datasets too. The generalized methodology may find application in addressing similar real-world problems from other domains too. There also exists scope of further acceleration by reducing device-host data transfer overheads. In this regard, other GPU compatible programming frameworks such as C/C++ OpenMP and OpenCL may be explored. A synchronous version of ABC algorithm should also contribute in further acceleration of the calibration process, provided quality of calibration results is not compromised. The work also provides insight for accelerated processing of multiple batches of *k*-NN queries in other domains.

Acknowledgments

The authors are grateful to Director SASE for permitting the use of data and compute facilities for this study. The contribution by Mr. Bhanu Damir in CUDA programming during the initial stages of this study is sincerely acknowledged. The MATLAB code of basic ABC algorithm used in this study was downloaded from <http://mf.erciyes.edu.tr/abc/software.htm>. Special thanks to anonymous reviewers and editors for critical review of the manuscript and many a useful suggestion.

References

- [1] F.S. Al-Qunaeir, H.R. Tizhoosh, S. Rahnamayan, Opposition based computing – a survey, in: IEEE International Joint Conference on Neural Networks, IJCNN, 2010, pp. 1–7.
- [2] A.S. Arefin, C. Riveros, R. Berretta, P. Moscato, GPU-FS-k NN: A software tool for fast and scalable k NN computation using GPUs, PLoS One 7 (8) (2012) e44000.
- [3] R.J. Barrientos, J.I. Gómez, C. Tenllado, M.P. Matias, M. Marin, kNN query processing in metric spaces using GPUs, in: Euro-Par 2011 Parallel Processing, Springer, Berlin, Heidelberg, 2011, pp. 380–392.
- [4] A. Basturk, R. Akay, Parallel implementation of synchronous type artificial bee colony algorithm for global optimization, J. Optim. Theory Appl. 155 (3) (2012) 1095–1104.

Table 10

Break-down of execution time of an iteration of model calibration process through CPU multi-core (combined) implementation with 12 cores. (FEs: Function evaluations).

Implementation type	Population size	Employed bee phase		Onlooker bee phase		FEs/Iteration	Compute time/Iteration	Overall acceleration
		FEs	Compute time	FEs	Compute time			
		A	B	C	D	E	F (B + D)	H
CPU-only multi-core (combined)	60	30	3.7302	30	5.2476	60	8.9806	2.86×

Table 11

Break-down of execution time of an iteration of model calibration process through CPU multi-core (combined) implementation for varying numbers of CPU cores used. (FEs: Function evaluations).

CPU cores used	Population size	Employed bee phase		Onlooker bee phase		FEs/Iteration	Compute time/Iteration	Overall acceleration
		FEs	Compute time	FEs	Compute time			
		A	B	C	D	E	F (B + D)	H
12	60	30	3.7302	30	5.2476	60	8.9806	2.86×
10	60	30	3.8015	30	5.5284	60	9.3305	2.76×
8	60	30	4.8837	30	6.2143	60	11.0983	2.32×
6	60	30	6.0680	30	7.9191	60	13.9876	1.84×
4	60	30	10.1684	30	11.0207	60	21.1895	1.21×
2	60	30	20.1087	30	20.7685	60	40.8778	0.63×

- [5] C. Blum, A. Roli, *Metaheuristics in combinatorial optimization: Overview and conceptual comparison*, ACM Comput. Surv. 35 (3) (2003) 268–308.
- [6] B. Brabec, R. Meister, A nearest-neighbour model for regional avalanche forecasting, Ann. Glaciol. 32 (2001) 130–134.
- [7] O. Buser, *Avalanche forecasting with method of nearest neighbours: An interactive approach*, Cold Reg. Sci. Technol. 8 (1983) 155–163.
- [8] O. Buser, M. Butler, W. Good, *Avalanche forecast by nearest-neighbour model*, IAHS Publ. 162 (1987) 557–569.
- [9] L. Cayton, Accelerating nearest neighbor search on manycore systems, in: IEEE 26th International Parallel & Distributed Processing Symposium, IPDPS, 2012, pp. 402–413.
- [10] J.M. Cecilia, J.M. García, A. Nisbet, M. Amos, M. Ujaldón, Enhancing data parallelism for ant colony optimization on GPUs, J. Parallel Distrib. Comput. 73 (1) (2013) 42–51.
- [11] D.J. Chang, A.H. Desoky, M. Ouyang, E.C. Rouchka, Compute pairwise Manhattan distance and Pearson correlation coefficient of data points with GPU, in: IEEE 10th ACIS International Conference on Software Engineering, Artificial Intelligences, Networking and Parallel/Distributed Computing, SNPD'09, 2009, pp. 501–506.
- [12] A. Delévacq, P. Delisle, M. Gravel, M. Krajecki, Parallel ant colony optimization on graphics processing units, J. Parallel Distrib. Comput. 73 (1) (2013) 52–61.
- [13] Q. Duan, S. Sorooshian, V. Gupta, Effective and efficient global optimization for conceptual rainfall-runoff models, Water Resour. Res. 28 (4) (1992) 1015–1031.
- [14] V. Garcia, E. Debreuve, M. Barlaud, Fast k nearest neighbor search using GPU, in: Proceedings of IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops, CVPRW'08, 2008, pp. 1–6.
- [15] V. Garcia, E. Debreuve, F. Nielsen, M. Barlaud, K-nearest neighbor search: Fast GPU-based implementations and application to high-dimensional feature matching, in: 17th IEEE International Conference on Image Processing, ICIP, 2010, pp. 3757–3760.
- [16] M. Gassner, B. Brabec, Nearest neighbour models for local and regional avalanche forecasting, Nat. Hazards Earth Syst. Sci. 2 (2002) 247–253.
- [17] M. Gassner, H. Etter, K. Birkeland, NXD2000 - An improved avalanche forecasting program based on the nearest neighbor method, in: Proceedings of International Snow Science Workshop, Big Sky, Montana, USA, 2000, pp. 52–59.
- [18] P.N. Glaskowsky, NVIDIA's Fermi: The First Complete GPU Computing Architecture, NVIDIA Corporation, 2009.
- [19] F. Glover, Future paths for integer programming and links to artificial intelligence, Comput. Oper. Res. 13 (4) (1986) 533–549.
- [20] A. Gómez-Iglesias, M.A. Vega-Rodríguez, F. Castejón, M. Cárdenas-Montes, E. Morales-Ramos, Artificial bee colony inspired algorithm applied to fusion research in a grid computing environment, in: IEEE 18th Euromicro International Conference on Parallel, Distributed and Network-Based Processing, PDP, 2010, pp. 508–512.
- [21] J. Heierli, R.S. Purves, A. Felber, J. Kowalski, Verification of nearest-neighbours interpretations in avalanche forecasting, Ann. Glaciol. 38 (2004) 84–88.
- [22] Y.S. Hong, Z.Z. Ji, C.L. Liu, Research of parallel artificial bee colony algorithm based on MPI, Appl. Mech. Mater. 380 (2013) 1430–1433.
- [23] R. Jovanovic, M. Tuba, I. Brajevic, Parallelization of the cuckoo search using cuda architecture, in: 7th International Conference on Applied Mathematics, Simulation, Modelling, ASM'13, 2013.
- [24] D. Karaboga, An idea based on honeybee swarm for numerical optimization, Technical Report TR06, Erciyes University, Engineering Faculty, Computer Engineering Department, 2005.
- [25] D. Karaboga, B. Basturk, A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm, J. Global Optim. 39 (2007) 459–471.
- [26] K. Kato, T. Hosino, Solving k-nearest neighbor problem on multiple graphics processors, in: Proc. of 10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing, 2010, pp. 769–773.
- [27] R. Kohavi, A study of cross-validation and bootstrap for accuracy estimation and model selection, in: Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence, San Mateo, CA, USA. Morgan Kaufmann, 2, (12), 1995, pp. 1137–1143.
- [28] K. Kristensen, C. Larsson, An avalanche forecasting Program based on a modified nearest neighbor method, in: Proceedings of International Snow Science Workshop, Snowbird, Utah, USA, 1994, pp. 22–30.
- [29] Q. Kuang, L. Zhao, A practical GPU based kNN algorithm, in: International Symposium on Computer Science and Computational Technology, ISCST, 2009, pp. 151–155.
- [30] W.B. Langdon, Graphics processing units and genetic programming: an overview, Soft Comput. 15 (8) (2011) 1657–1669.
- [31] S. Li, N. Amenta, Brute-force k-nearest neighbors search on the GPU, in: Similarity Search and Applications, Springer International Publishing, 2015, pp. 259–270.
- [32] J. Li, X. Hu, Z. Pang, K. Qian, A parallel ant colony optimization algorithm based on fine-grained model with GPU-acceleration, Int. J. Innovative Comput. Inf. Control 5 (11) (2009) 3707–3716.
- [33] S. Liang, C. Wang, Y. Liu, L. Jian, CUKNN: A parallel implementation of K-nearest neighbor on CUDA-enabled GPU, in: Proc. of IEEE Youth Conference on Information, Computing and Telecommunication, YC-ICT'09, 2009, pp. 415–418.
- [34] G.H. Luo, S.K. Huang, Y.S. Chang, S.M. Yuan, A parallel bees algorithm implementation on GPU, J. Syst. Archit. 60 (3) (2014) 271–279.
- [35] D. McClung, M. Tweedy, Numerical avalanche prediction: Kootney Pass, British Columbia, Can. J. Glaciol. 40 (135) (1994) 350–358.
- [36] C. McCollister, K. Birkeland, K. Hansen, R. Aspinall, R. Comey, A probabilistic technique for exploring multi-scale spatial patterns in historical avalanche data by combining GIS and meteorological nearest neighbours with an example from the Jackson Hole Ski Area, Wyoming, in: Stevens, J.R. (Ed.), Proceedings of International Snow Science Workshop, Penticton, BC, Canada, 2002.
- [37] D.M. Muñoz, C.H. Llanos, L.D.S. Coelho, M. Ayala-Rincón, Accelerating the artificial bee colony algorithm by hardware parallel implementations, in: IEEE Third Latin American Symposium on Circuits and Systems, LASCAS, 2012, pp. 1–4.
- [38] L. Mussi, F. Daolio, S. Cagnoni, Evaluation of parallel particle swarm optimization algorithms within the CUDA™ architecture, Inform. Sci. 181 (20) (2011) 4642–4657.
- [39] H. Narasimhan, Parallel artificial bee colony (PABC) algorithm, in: Congress on Nature & Biologically Inspired Computing, Coimbatore, India, 2009, pp. 306–311.
- [40] NVIDIA, CUDA C Best Practices Guide, v. 6.5, NVIDIA Corporation, 2014.
- [41] NVIDIA, CUDA C Programming Guide, v. 6.5, NVIDIA Corporation, 2014.
- [42] NVIDIA, Whitepaper - NVIDIA's Next Generation CUDA Compute Architecture: Fermi, NVIDIA Corporation, 2009.
- [43] C. Obled, W. Good, Recent developments of avalanche forecasting by discriminant analysis techniques: a methodological review and some applications to the Parsenn area (Davos, Switzerland), J. Glaciol. 25 (92) (1980) 315–346.
- [44] J. Pan, D. Manocha, Bi-level locality sensitive hashing for k-nearest neighbor computation, in: IEEE 28th International Conference on Data Engineering, ICDE, 2012, pp. 378–389.
- [45] P. Pospichal, J. Jaros, J. Schwarz, Parallel genetic algorithm on the cuda architecture, in: Applications of Evolutionary Computation, Springer, Berlin, Heidelberg, 2010, pp. 442–451.

- [46] R.S. Purves, K.W. Morrison, G. Moss, D.S.B. Wright, Nearest neighbours for avalanche forecasting in Scotland - development, verification and optimisation of a model, *Cold Reg. Sci. Technol.* 37 (2003) 343–355.
- [47] J. Reese, S. Zaranek, *GPU Programming in MATLAB*, 91967v01, The MathWorks, Inc., 2012.
- [48] N. Satish, M. Harris, M. Garland, Designing efficient sorting algorithms for manycore GPUs, in: *Proc. of the IEEE International Symposium on Parallel & Distributed Processing*, (IPDPS'09), IEEE Computer Society, Washington, DC, USA, 2009, pp. 1–10.
- [49] A. Singh, B. Damir, K. Deep, A. Ganju, Calibration of nearest neighbors model for avalanche forecasting, *Cold Reg. Sci. Technol.* 109 (2015) 33–42.
- [50] A. Singh, A. Ganju, Artificial Neural Networks for Snow Avalanche Forecasting in Indian Himalaya, in: *Proceedings of 12th International Conference of International Association for Computer Methods and Advances in Geomechanics, IACMAG*, 1–6 October 2008, Goa, India, 2008.
- [51] A. Singh, K. Srinivasan, A. Ganju, Avalanche forecast using numerical weather prediction in Indian Himalaya, *Cold Reg. Sci. Technol.* 43 (2005) 83–92.
- [52] N. Sismanis, N. Pitsianis, X. Sun, Parallel search of k-nearest neighbors with synchronous operations, in: *IEEE Conference on High Performance Extreme Computing*, HPEC, 2012, pp. 1–6.
- [53] K. Sørensen, F.W. Glover, Metaheuristics, in: *Encyclopedia of Operations Research and Management Science*, Springer, US, 2013, pp. 960–970.
- [54] M. Subotic, M. Tuba, N. Stanarevic, Parallelization of the artificial bee colony (ABC) algorithm, in: *Proceedings of the 11th WSEAS International Conference on Neural Networks and 11th WSEAS International Conference on Evolutionary Computing and 11th WSEAS International Conference on Fuzzy Systems*, World Scientific and Engineering Academy and Society (WSEAS), Stevens Point, Wisconsin, USA, NN, 2010, pp. 191–196.
- [55] M. Subotic, M. Tuba, N. Stanarevic, Different approaches in parallelization of the artificial bee colony algorithm, *Int. J. Math. Models Methods Appl. Sci.* 5 (4) (2011) 755–762.
- [56] L. Sun, C. Stoller, T. Newhall, Hybrid MPI and GPU approach to efficiently solving large kNN problems. Tera Grid Poster, 2010. URL <http://archive.isgtw.org/pdfs/kNNposter.pdf>.
- [57] G. Teodoro, E. Valle, N. Mariano, R. Torres, W. Meira Jr., J.H. Saltz, Approximate similarity search for online multimedia services on distributed CPU–GPU platforms, *VLDB J.* 23 (3) (2014) 427–448.
- [58] D.S. Wilks, *Statistical Methods in the Atmospheric Sciences: An Introduction*, Academic Press, San Diego, 1995.
- [59] Y. Zhou, Y. Tan, GPU-based parallel particle swarm optimization, in: *IEEE Congress on Evolutionary Computation*, CEC'09, 2009, pp. 1493–1500.



Amreek Singh is working as scientist at Snow & Avalanche Study Establishment (SASE), Chandigarh (India) since 1997. His research areas include applications of statistics, soft computing, nature-inspired metaheuristics, data mining, feature ranking and geospatial techniques for mountain weather and avalanche forecasting. He developed and operationally implemented *k*-nearest neighbours method based decision support system 'eNN10' for avalanche forecast in Indian Himalaya. He is life member of Indian Meteorological Society, Indian Society of Remote Sensing and Soft Computing Research Society.



Kusum Deep is with Department of Mathematics, Indian Institute of Technology–Roorkee, Roorkee (India). She earned her Ph.D. from IIT Roorkee in 1988. She is a Senior Member of Operations Research Society of India, IEEE, Computer Society of India, Indian Mathematical Society, and Indian Society of Industrial Mathematics. Besides being on the editorial board of a number of journals of repute, she is Executive Editor of *International Journal of Swarm Intelligence* and an expert on the panel of Department of Science & Technology, Govt. of India. Her research interests are Nature Inspired Optimization Techniques and their applications.



Pallavi Grover is currently working at Snow & Avalanche Study Establishment (SASE), Chandigarh (India) as a research fellow and also pursuing Ph.D. from Panjab University, Chandigarh (India). She completed M.Sc. (Hons) in Computer Science and Application from Panjab University, Chandigarh in 2012. Her areas of interest include Semantic Web, Machine Learning and Data Mining.